Highlights

Process Mining and Stochastic Modeling for Software Process Forecasting: A Systematic Literature Review with an LLM-Assisted Protocol

Rodrigo Almeida de Oliveira, Juliano de Paulo Ribeiro, Edson Emilio Scalabrin

- LLM-assisted SLR protocol screens 6,147 software-process papers across two corpus tiers.
- Cross-model Cohen's $\kappa = 0.695$ inter-rater agreement on a 20% stratified sample.
- 404 primary studies confirmed; only 1 jointly satisfies PM, stochastic, and forecasting.
- Five findings (F1–F5) and SPMF taxonomy proposed; four-direction research agenda.
- Full replication package (prompts, scripts, decisions) deposited at Zenodo.

Process Mining and Stochastic Modeling for Software Process Forecasting: A Systematic Literature Review with an LLM-Assisted Protocol

Rodrigo Almeida de Oliveira[a,1,*], Juliano de Paulo Ribeiro[a,2], Edson Emilio Scalabrin[a,3]

[a]*Pontifícia Universidade Católica do Paraná (PUCPR), Graduate Program in Informatics (PPGIa), Curitiba, PR, Brazil*

Abstract

Context. Software development processes generate rich event data — commits, issues, pull requests, CI/CD logs — that process mining (PM) and stochastic models can analyze to support forecasting. Yet integration of these techniques for software development lifecycle (SDLC) prediction is reported only anecdotally. **Objective.** To map the state of the art at the intersection of process mining, stochastic modeling, and machine-learning forecasting applied to software processes; to identify research gaps; and to propose a research agenda. **Method.** We executed a systematic literature review following Kitchenham and Charters (2007), Petersen et al. (2015) for mapping, and Wohlin (2014) for snowballing, augmented with an LLM-assisted screening protocol (*claude-haiku-4-5* as primary screener, *claude-sonnet-4-6* as cross-model verifier). From 5,783 deduplicated records plus 364 control and manually imported items, the protocol screened two corpus tiers: a 2,340-paper working-set tier and a 3,807-paper auxiliary tier (6,147 papers screened in total). Cohen's κ inter-rater agreement was computed on stratified 20% samples; 8-criterion quality assessment (Dyba and Dingsoyr, 2008) was applied to all confirmed includes. **Results.** 404 primary studies were confirmed (169 working-set + 212 first-pass auxiliary + 23 second-pass auxiliary after abstract enrichment). Inter-rater agreement was substantial in the working-set tier (κ_{binary} = 0.695 T/A, 0.694 FT). Five findings emerge: F1 a descriptive ceiling (PM dominant, forecasting minority); F2 ad-hoc event-log construction; F3 weak stochastic–PM integration; F4 pipeline fragmentation (only 1 of 404 papers jointly satisfies PM + stochastic + forecasting); F5 process-derived ML features unexplored. **Conclusion.** The literature is descriptively rich but predictively fragmented. We propose the SPMF taxonomy and a four-direction research agenda; PATHCAST is introduced as the first formally specified L3 pipeline for SDLC forecasting.

Keywords: Systematic Literature Review, Process Mining, Stochastic Modeling, Software Process Forecasting, LLM-Assisted Screening, Mining Software Repositories

*Corresponding author.

Email addresses: rodrigo1.almeida@pucpr.edu.br (Rodrigo Almeida de Oliveira), juliano.ribeiro@pucpr.edu.br (Juliano de Paulo Ribeiro), scalabrin@ppgia.pucpr.br (Edson Emilio Scalabrin)

[1]ORCID: 0009-0009-6310-4126

[2]ORCID: 0009-0004-3605-485X

[3]ORCID: 0000-0002-3918-1799

1. Introduction

This article reports a systematic literature review (SLR) conducted to map the state of the art at the intersection of process mining, stochastic modeling, and machine learning applied to software process forecasting. The review follows the guidelines established by Kitchenham and Charters [1] for evidence-based software engineering, complemented by the snowballing procedures described by Wohlin [2] and the mapping study guidelines of Petersen et al. [3]. The primary objective is to identify, classify, and synthesize existing approaches that combine process-aware techniques with predictive modeling for software development outcomes, establishing the research gap that motivates PATHCAST — a proposed end-to-end pipeline composing process mining, absorbing Markov chain analysis, Monte Carlo forecasting, and machine-learning enhancement for software development lifecycle (SDLC) data, sketched briefly in Section 5.2.

The review addresses three overarching research questions, each decomposed into two sub-questions covering the application landscape of process mining in SDLC contexts, the techniques and prediction methods employed, and the degree of integration between process mining and stochastic forecasting methods.

2. Review Protocol

2.1. Research Questions

The SLR is guided by the following research questions, structured to progressively map the landscape from application scope to technique integration:

Table 1: Research questions and expected outputs.

RQ	Sub-RQ	Question	Expected Output
RQ1	RQ1.1	Which SDLC phases have been analyzed with process mining?	Frequency distribution by phase
RQ1	RQ1.2	What event log sources are used and how are they constructed?	Taxonomy of sources and construction methods
RQ2	RQ2.1	Which process discovery algorithms have been applied to SDLC?	Comparative matrix
RQ2	RQ2.2	Which predictive and stochastic techniques have been used?	Technique classification
RQ3	RQ3.1	What is the level of integration between PM and stochastic methods?	Integration maturity assessment
RQ3	RQ3.2	What gaps exist for process-aware forecasting in the SDLC?	Gap analysis matrix

2.2. Search Strategy

The search strategy combines automated database search with backward and forward snowballing. The automated search targets five digital libraries: IEEE Xplore, ACM Digital Library, Scopus, Web of Science, and SpringerLink.

2.2.1. PICO Decomposition

The search string construction follows the PICO framework adapted for software engineering, as shown in Table 2.

Table 2: PICO decomposition for search string construction.

Facet	**Role**	**Concepts**
Population	Application domain	Software engineering, SDLC, software process, DevOps, CI/CD
Intervention	Analytical technique	Process mining, Markov chain, Monte Carlo, predictive PM, ML
Comparison	N/A	(omitted)
Outcome	Result	Forecasting, prediction, lead time, cycle time, process model

2.2.2. Search Strings

Three complementary search strings are executed to maximize coverage while maintaining precision.

Principal String ($P \wedge I$).. Block P captures the software engineering context:

```
("software engineering" OR "software development" OR "software process" OR "software lifecycle"
OR "software life cycle" OR "SDLC" OR "development lifecycle" OR "development life cycle"
OR "DevOps" OR "CI/CD" OR "continuous integration" OR "continuous delivery" OR "agile development"
OR "agile process" OR "issue tracking" OR "bug tracking" OR "version control" OR "code review"
OR "pull request" OR "software project" OR "software repository" OR "software maintenance"
OR "software evolution")
```

Block I captures the process mining and stochastic modeling techniques:

```
("process mining" OR "process discovery" OR "conformance checking" OR "workflow mining" OR
"predictive process monitoring" OR "process prediction" OR "remaining time prediction" OR
"process forecasting" OR "event log" OR "process simulation" OR "Monte Carlo simulation" OR
"Markov chain" OR "Markov model" OR "stochastic process model" OR "transition probability"
OR "absorbing chain" OR "lead time prediction" OR "cycle time prediction")
```

Complementary String (MSR → Process Modeling).. Captures studies using MSR terminology without canonical process mining terms:

```
("mining software repositories" OR "software repository mining" OR "GitHub mining" OR "Jira
mining") AND ("process model" OR "Markov" OR "Monte Carlo" OR "stochastic" OR "lead time"
OR "cycle time" OR "throughput" OR "transition matrix")
```

Frontier String (Stochastic ∧ PM ∧ SE).. Targets the intersection of stochastic methods with process mining in software contexts:

```
("Monte Carlo" OR "Markov chain" OR "stochastic simulation" OR "probabilistic forecast" OR
"transition matrix") AND ("process mining" OR "event log" OR "workflow mining" OR "process
model" OR "business process") AND ("software development" OR "software engineering" OR "software
process" OR "DevOps" OR "SDLC" OR "issue tracker" OR "commit log" OR "build pipeline")
```

The strings are adapted to each database's syntax and field conventions as summarized in Table 3. The search scope covers publications from January 1994 to December 2026.

Table 3: Database-specific search adaptations and raw retrieval counts.

Database	**Search Field**	**Strings**	**Filters**	**Records**
Scopus	TITLE-ABS-KEY	3	ar, cp; EN; 1994–2026	2,377
IEEE Xplore	All Metadata	4 (split)	Conf. + Journals; 1994–2026	1,386
Web of Science	TS (Topic)	3	Article, Proc.; EN; 1994–2026	997
SpringerLink	Simplified	8 queries	Chapter, Article; EN	2,884
ACM DL	Abstract	2	Date: 1994–2026	96
Snowballing	–	–	Backward + forward	595
Control set	–	–	Manual	12
Total				**8,347**

2.2.3. Snowballing Procedure

Backward snowballing examines the reference lists of all included primary studies. Forward snowballing uses Google Scholar citation tracking to identify subsequent works that cite the included studies. Both rounds apply the same inclusion and exclusion criteria defined in Section 2.3.

2.2.4. Search String Validation

The search strings are validated against a control set of 10 papers that are known to be relevant to the review scope (Table 4). The acceptance threshold is $\geq 8/10$ papers found; the validation returned $10/10$ (100%, PASS).

Table 4: Control papers for search string validation.

#	Paper	Justification	Captured via
V1	Cook & Wolf (1998)	Seminal PM + SE	ACM
V2	Rubin et al. (2007)	PM framework for SW processes	Scopus
V3	Poncin et al. (2011)	PM + MSR intersection	IEEE
V4	Caldeira & Cardoso (2019)	PM + SW process assessment	IEEE
V5	Lemos et al. (2011)	PM for SW process management	IEEE
V6	Whittaker & Thomason (1994)	Markov + SW testing (seminal)	Scopus
V7	Tax et al. (2017)	PPM seminal (BPM baseline)	Control
V8	Rogge-Solti & Weske (2015)	Stochastic + PM forecasting	Scopus
V9	Rozinat et al. (2009)	PM → Simulation bridge	Control
V10	Gupta & Sureka (2017)	PM multi-repository in SE	ACM

2.3. Inclusion and Exclusion Criteria

The inclusion criteria are organized in two tiers. Formal criteria (IC1–IC3) filter on publication attributes and are applied during the database search configuration. Content criteria (IC4a–IC4d) require that the study's research contribution addresses at least one of four specific technique-domain intersections; they are applied during title/abstract and full-text screening. Exclusion criteria (EC1–EC5) are applied cumulatively across both screening phases. Criteria EC1–EC4 are evaluated by the LLM-assisted screener; EC5 (full text inaccessible) is applied programmatically after the inter-library-loan recovery pass for Band D/E papers fails to retrieve a copy. In the auxiliary tier, EC5 is extended to cover non-recoverable metadata insufficiency: papers for which no usable abstract could be recovered after the multi-source enrichment cascade, and papers for which the primary and verifier raters produced an irreconcilable decision without accessible full text, are closed under EC5 rather than escalated to human arbitration.

At least one of IC4a–IC4d must be satisfied for inclusion. Criterion IC4b ensures that seminal works applying Markov chains to software processes—such as Whittaker and Thomason [4]—are captured despite predating the term "process mining."

2.4. LLM-Assisted Screening Methodology

Given the scale of the working set (2,340 papers) and the requirement for systematic, reproducible screening decisions with traceable rationale, the title/abstract screening phase employs a language model as the primary screener. Specifically, *claude-haiku-4-5-20251001* is used via the Anthropic Message Batches API, with a structured prompt

Table 5: Inclusion and exclusion criteria.

Type	ID	Criterion
Inclusion	IC1	Publication period: 1994–2026 (from Whittaker & Thomason onward).
	IC2	Language: English.
	IC3	Peer-reviewed journal article, conference paper, or workshop paper (≥4 pages).
	IC4a	Applies process mining techniques (discovery, conformance checking, predictive process monitoring) to software development artifacts (commits, issues, pull requests, CI/CD logs, bug trackers, VCS).
	IC4b	Uses stochastic models (Markov chains, Monte Carlo, stochastic Petri nets, absorbing chains) to analyze software development processes or workflows.
	IC4c	Proposes or evaluates forecasting of software process metrics (lead time, cycle time, remaining time, throughput, defect rates) from event logs or repository data.
	IC4d	Mines software repositories (GitHub, Jira, VCS, CI/CD platforms) to discover, analyze, or improve software process models.
Exclusion	EC1	Application domain is exclusively outside software development (healthcare, manufacturing, supply chain) without demonstrated relevance to SDLC processes.
	EC2	Software appears only as the implementation platform; the process being analyzed is not a software development process.
	EC3	Purely algorithmic or theoretical contribution without empirical evaluation in a software development context.
	EC4	Secondary study (SLR, mapping study, survey) not specifically addressing PM or stochastic methods in software engineering.
	EC5	Full text or decision-critical metadata inaccessible despite all retrieval attempts (open access, institutional, ILL, author-provided preprint, multi-source abstract enrichment). Applied programmatically after Band D/E recovery and auxiliary enrichment; also closes irreconcilable ambiguous cases when no accessible full text exists.

encoding the IC4a–IC4d and EC1–EC4 criteria (EC5 is applied programmatically post-screening), the PATHCAST scope description, and decision rules for the three-class output: `include`, `exclude`, or `maybe`.

The screening prompt instructs the model to (i) identify which ICs are satisfied, (ii) identify which ECs apply, (iii) assign a decision with a one-to-two sentence rationale, and (iv) return the result as structured JSON. The `maybe` class captures genuine uncertainty—typically arising from missing abstracts or ambiguous domain terminology—without forcing a binary decision.

At the full-text screening phase, the same model is applied with a stricter prompt that enforces a binary outcome (`include` or `exclude`) with `pending` reserved only when the abstract is genuinely insufficient for a decision.[4]

This approach extends prior work on LLM-assisted systematic reviews [5, 6] to the software process mining domain. The primary screener produces structured outputs that enable post-hoc quality analysis. Validation follows a two-stage protocol: (i) all `include` decisions are manually verified by the author; (ii) a stratified random 20% sample of T/A and FT decisions is independently re-rated by a different model family (*claude-sonnet-4-6*) acting as a second rater, and inter-rater agreement is reported via Cohen's κ following the Landis and Koch (1977) interpretation thresholds. The substantial-agreement threshold ($\kappa \geq 0.61$) is the target reported by Khraisha et al. [6] for LLM-assisted screening in the medical SLR domain and adopted here for SE-domain SLRs. Cross-model verification is recommended by Syriani et al. [5] as a practical proxy when a fully independent human second rater is not feasible at the working-set scale considered here. Results of this verification are reported in Section 6 (Internal Validity).

Operationally, the protocol does not preserve an open-ended manual-arbitration queue. If a paper still lacks decision-critical evidence after the retrieval and enrichment cascade, or if the verifier can only return `pending` because the metadata remain insufficient, the paper is closed under EC5 and excluded from the valid evidence set.

2.5. Quality Assessment

Each included study is assessed against a quality checklist adapted from Dybå and Dingsøyr [7], with eight binary criteria:

Studies scoring below 4.0/8 (50%) are excluded from the synthesis. QA8 reflects the centrality of process model quality in the PATHCAST pipeline.

QA scoring is performed by the same LLM-assisted protocol used for screening (Section 2.4): *claude-haiku-4-5-20251001* receives a strict rubric for each criterion and returns a binary score with a brief justification. The full prompt and the per-study scores are part of the replication package (Section 6.5). Table 7 reports the quality-assessment outcome for the complete confirmed set ($n = 169$). All 169 studies were scored.

[4] The two protocols use different nomenclature for the uncertainty class: T/A uses `maybe` (epistemically broad: criterion ambiguity *or* abstract insufficiency), whereas FT uses `pending` (narrowly: abstract insufficiency only). The semantic difference is intentional — the T/A pass operates on metadata where ambiguity is expected, while the FT pass commits to binary decisions whenever the abstract permits and escalates only the abstract-insufficient subset to manual full-text retrieval. This mapping is preserved in the cross-model κ analysis (Tables 14 and 15).

Table 6: Quality assessment checklist.

#	Criterion	Score
QA1	Are the research objectives clearly stated?	0/1
QA2	Is the software engineering context described in detail?	0/1
QA3	Is the data source (event log) described reproducibly?	0/1
QA4	Is the PM or stochastic technique formally defined?	0/1
QA5	Are results validated empirically?	0/1
QA6	Are threats to validity discussed?	0/1
QA7	Is the study reproducible (data and/or code available)?	0/1
QA8	Are process model quality metrics reported (fitness, precision)?	0/1

Table 7: Quality-assessment results for the confirmed included set (n = 169; full QA applied). Auto-generated from `pipeline/qa_llm.py`.

QA Outcome	Count	Notes
Studies assessed	169	Absolute count and 100.0% of included set
Studies with score $\geq 4/8$	136	Retained for synthesis
Studies with score $< 4/8$	33	Excluded after QA
Mean QA score	4.88	Mean and standard deviation (1.40)
Median QA score	5.00	Median and interquartile range (4.00–6.00)

The 33 studies below the QA threshold are predominantly short workshop papers, position papers, or extended abstracts that fail QA3 (reproducible data source) and QA4 (formally defined technique) simultaneously. They are not removed from the PRISMA flow; they are retained in the included set as context but are not used in the F1–F5 evidence base reported in Section 5.1.

2.6. Data Extraction

For each included study, the attributes in Table 8 are extracted.

Table 8: Data extraction form.

Field	**Description**
ID	Unique identifier
Authors, Year, Venue	Bibliographic metadata
SDLC Phase(s)	Lifecycle phases covered
Event Log Source	Jira, GitHub, GitLab, Jenkins, etc.
Event Log Construction	Manual, semi-automatic, or automatic
PM Technique Category	Discovery / Conformance / Enhancement / Prediction
Specific Algorithm(s)	Alpha, Heuristics, Inductive Miner, etc.
Stochastic Method	Markov / Monte Carlo / Stochastic Petri Net / None
ML Technique	RF, XGBoost, LSTM, etc. / None
Prediction Target	Remaining time, outcome, lead time, delay, etc.
Integration Level	L0 None / L1 Sequential / L2 Pipeline / L3 Unified
Validation Type	Case study / Experiment / Industrial / Simulation
Tool/Platform	PM4Py, ProM, Disco, custom, etc.
Dataset Size	Number of events and cases analyzed
Process Model Quality	Fitness, precision, generalization (if reported)

3. Search Results and Study Selection

3.1. Database Retrieval and Deduplication

The three search strings executed across five databases returned 7,740 records. Backward and forward snowballing from the control set yielded an additional 595 records. Together with 12 control papers maintained as a validation-only set, the initial corpus totaled 8,347 records. Table 9 summarizes the retrieval by source.

Cross-source deduplication removed 2,564 duplicate records (30.7%), yielding 5,781 unique primary-corpus papers plus 2 unique control papers. Deduplication used normalized DOI matching, followed by fuzzy title matching (Levenshtein similarity ≥ 0.9) for records without DOIs.

Table 9: Record retrieval by source.

Source	Raw Records	After Internal Dedup
Scopus (6 queries)	2,377	1,852
SpringerLink (8 queries, 10 export batches)	2,884	1,543
IEEE Xplore (manual)	1,386	1,263
Snowballing	595	575
Web of Science (4 queries)	997	514
ACM Digital Library (3 queries)	96	34
Control set (validation only)	12	2
Total	**8,347**	**5,783**

3.2. Operational Working Set Construction

Screening 5,783 records in a single pass would require substantial resources. A working set of 2,340 papers was therefore constructed from the highest-signal primary queries—those whose query design directly targets the PATHCAST intersection (process mining, stochastic modeling, and software development)—as shown in Table 10. The working-set construction also incorporates 364 additional records from the manually imported control set and supplementary IEEE/ACM batches that were not part of the primary deduplication pool. The 3,807 unique records not selected for the working set therefore constitute the *high-recall auxiliary corpus*, retained as a frozen baseline and available for supplementary analysis if gaps in the primary results warrant expansion.[5]

Table 10: Working set composition.

Database	Queries included	Unique papers
Scopus	scopus_principal, scopus_complementar, scopus_fronteira, scopus_markov_testing, scopus_stochastic_petri, scopus_simulation_bridge	2,197
IEEE Xplore	ieee_manual_01 (PM + SE subgroup)	97
ACM DL	acm_principal, acm_msr	46
Total		**2,340**

The working set preserves 10/10 control papers when the validation-only control set is included, and 8/10 from primary queries alone—both above the 80% threshold.

[5]Composition: 5,783 unique deduplicated records, of which 1,976 were selected into the working set, plus 364 control and manually imported records added directly to the working set, totalling 2,340 working-set papers; the remaining 3,807 unique records constitute the auxiliary corpus.

3.3. Title/Abstract Screening

All 2,340 working-set papers were screened against IC4a–IC4d and EC1–EC4 using the LLM-assisted protocol described in Section 2.4. The screening was executed across five sequential batches of up to 500 papers each via the Anthropic Batches API, producing structured JSON outputs with decision, rationale, and matched criteria for each record.

Table 11: Title/abstract screening results.

Decision	**Count**	**%**
Include	111	4.7
Maybe	775	33.1
Exclude	1,454	62.2
Total	**2,340**	100.0

The `include` set (111 papers) and the `maybe` set (775 papers) together form the full-text review queue of 886 papers. The `exclude` decisions were driven primarily by EC1 (domain outside software development, principally generic business process management) and EC3 (theoretical algorithm proposals without SE evaluation).

3.4. Full-Text Screening

The 886 papers forwarded from the T/A phase were subjected to LLM-assisted full-text screening using the strict prompt described in Section 2.4 (decisions limited to `include`, `exclude`, or `pending`; `maybe` is not permitted at this stage). To maximise the number of papers assessable by the LLM, a multi-source abstract-enrichment cascade was executed prior to screening—combining DOI-based lookups via Semantic Scholar, OpenAlex, CORE, and Crossref, followed by title-based fallback lookups—raising abstract coverage from 16.6% to 72.6% (643 of 886 papers).[6]

Table 12 summarises the outcome.

Among the 717 excluded papers, EC3 (algorithmic or theoretical contribution without SE evaluation, $n = 232$) and EC1 (domain outside software development, $n = 205$) jointly account for the largest share of FT-stage exclusions. EC5 (full text inaccessible despite all retrieval attempts, active post-audit $n = 145$) corresponds to the residual portion of the original 148-paper Band D/E stratum for which no open-access copy, institutional access, or author-provided preprint could be located; these are recorded in a dedicated traceability file and documented in the PRISMA flow as a named exclusion stratum. EC2 (software as implementation platform rather than analyzed process, $n = 110$) and EC4 (out-of-scope secondary studies, $n = 60$) account for the remainder.

Figure 1 presents the complete PRISMA 2020 flow diagram [8] of the selection process.

[6] Papers were processed in order of a five-band priority scheme: Band A (91 papers, T/A `include` ≥2 ICs), Band B (20, T/A `include` 1 IC), Band C (39, T/A `maybe` + abstract), Band D (152, T/A `maybe` + IC signal, no abstract), Band E (584, T/A `maybe`, no IC signal, no abstract). Abstract enrichment and LLM re-screening were applied to Bands A, B, C, and the subset of D/E that gained an abstract after enrichment.

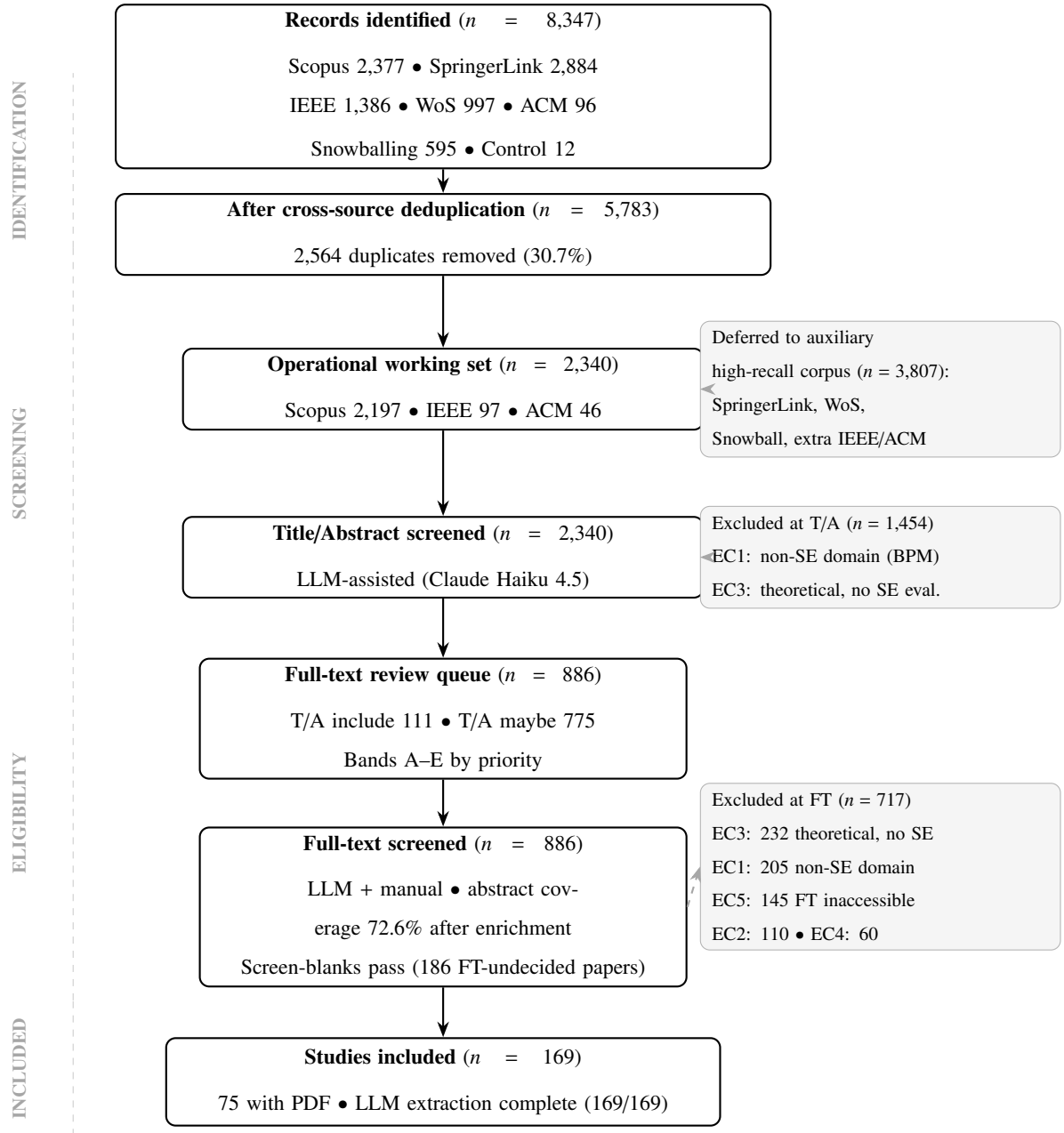

Figure 1: PRISMA 2020 flow diagram of the study selection process. Phase labels follow the four-stage structure: Identification, Screening, Eligibility, and Included.

Table 12: Full-text screening outcomes (886 papers forwarded from T/A phase).

Outcome	**Count**	**%**
Included	169	19.1
Excluded	717	80.9
EC3 (algorithmic, no SE eval.)	(232)	(26.2)
EC1 (non-SE domain)	(205)	(23.1)
EC5 (full text inaccessible, post-audit active stratum)	(145)	(16.4)
EC2 (SW as implementation)	(110)	(12.4)
EC4 (out-of-scope secondary)	(60)	(6.8)
Total	**886**	100.0

Note: EC counts are non-exclusive — a single paper may trigger multiple ECs when more than one applies (e.g., a non-SE-domain paper whose contribution is purely theoretical). The EC5 count of 145 is the active post-audit stratum after re-screening the original historical EC5 band of 148 papers and reclassifying 3 of them to EC3. The sum of per-EC counts is 232 + 205 + 145 + 110 + 60 = 752, which exceeds the 717 excluded papers by 35 (corresponding to papers triggering more than one EC). The percentages above are with respect to the 886-paper FT queue.

4. Results and Synthesis

4.1. Overview of Included Studies

The SLR is reported in two complementary tiers: (i) the **working-set tier**, comprising the 2,340-paper operational corpus that was deduplicated and processed through the LLM-assisted T/A and FT screening, yielding **169 confirmed primary studies**; and (ii) the **combined tier**, which adds the 3,807-paper deferred auxiliary corpus after a full T/A and FT pass on the same protocol, yielding 212 additional confirmed studies in the first auxiliary pass, plus 23 further confirmations from a second auxiliary pass over abstracts recovered through post-hoc enrichment (Section 6, External Validity), for a combined total of **404 confirmed primary studies** ($169_{\text{ws}} + 212_{\text{aux1}} + 23_{\text{aux2}}$). The working-set tier is the narrative spine of this article; the combined-tier numbers are reported in parallel for findings F1–F5 and provide the recall-bounded view of the SLR. Unless stated otherwise, detailed combined-tier distributions refer to the **381-study analytical subset** frozen before the final +23 auxiliary re-FT confirmations. The 169-study set spans three decades (1995–2026), with a clear acceleration from 2014 onward (Figure 2). Seventy-one percent of confirmed papers (working-set tier) were published between 2015 and 2026, reflecting the maturation of open-source process mining toolkits (PM4Py, ProM) and the broader adoption of deep learning frameworks in software engineering research.

By document type, the majority are conference papers and journal articles. By source database, Scopus is the dominant contributor, followed by IEEE Xplore and ACM DL. Data extraction was completed for all 169 studies using a structured LLM-assisted pipeline (*claude-haiku-4-5-20251001*) operating on locally available PDFs (75/169, of which 73 were operationally coded in the triangulation subset) or abstract/title metadata (94/169).

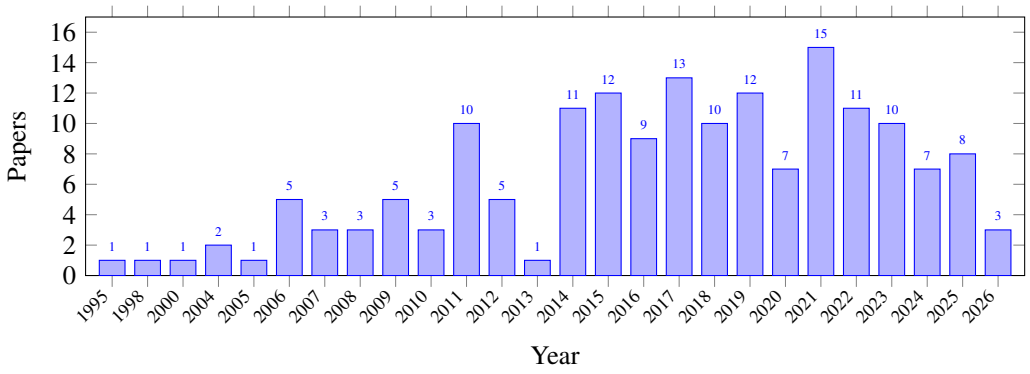

Figure 2: Distribution of confirmed studies by publication year (n = 169).

4.2. Inclusion Criterion Coverage

Figure 3 shows the distribution of inclusion criteria among the 169 confirmed papers (criteria are non-exclusive; a paper may satisfy multiple).

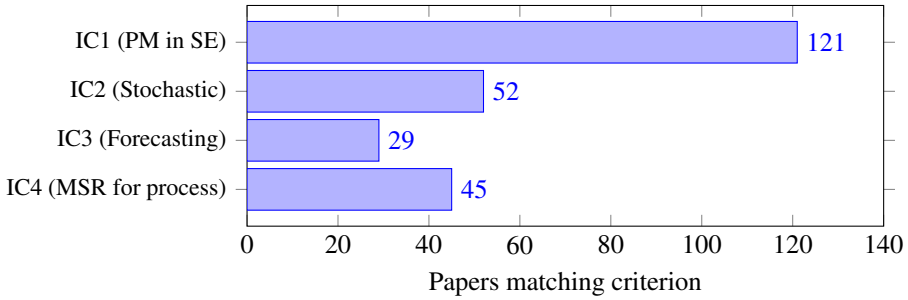

Figure 3: Inclusion criteria matched by the confirmed set (n = 169; non-exclusive; IC labels map to IC4a–IC4d in the protocol).

IC1/IC4a (process mining applied to SE artifacts) is the dominant criterion, matched by 121 papers (71.6%). IC4/IC4d (repository mining for process modeling) is second at 45 papers (26.6%), reflecting the MSR community's overlapping contributions. IC2/IC4b (stochastic modeling) appears in 52 papers (30.8%) and IC3/IC4c (forecasting of process metrics) in 29 papers (17.2%). Notably, 68 papers (40.2%) match IC1 only, without stochastic or forecasting components, providing direct empirical evidence for the "Descriptive Ceiling" identified in Finding F1 below.

4.3. Prevalent Exclusion Patterns

Among the 717 FT-excluded papers (Table 12), the largest exclusion reason is EC3 (algorithmic or theoretical contribution without software-engineering evaluation, n = 232), indicating a substantial body of process mining and stochastic modeling research that has not been empirically applied to software engineering contexts. EC1 (domain exclusively outside software development, n = 205) is the second-largest reason and consists primarily of generic business process management (BPM) studies that use the same terminology as the search strings but apply techniques

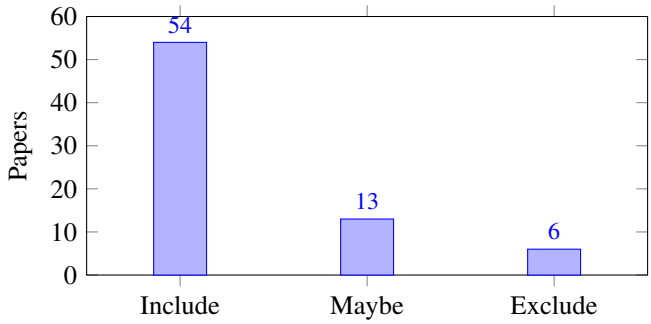

Figure 4: Operational decision distribution in the 73-paper PDF triangulation subset.

to organizational processes unrelated to SDLC. EC5 (full text inaccessible, active post-audit $n = 145$; original historical stratum $n = 148$) closes the group of large strata, followed by EC2 (software as implementation platform rather than analyzed process, $n = 110$) and EC4 (out-of-scope secondary study, $n = 60$). EC counts are non-exclusive: the sum of per-EC counts (232 + 205 + 145 + 110 + 60 = 752) exceeds the 717 excluded papers by 35, corresponding to papers that triggered more than one EC simultaneously. The dominance of EC3 and EC1 directly supports Findings F3 and F4 below: process mining and stochastic methods are widely studied in the abstract or in non-SE domains, but their application to SDLC processes remains comparatively narrow.

4.4. Full-Text Extraction and PDF-Based Triangulation

Structured data extraction was performed on the full 169-study set using the extraction template defined in Table 8. For each study, the LLM extracted the research question, study type, research contribution, PM technique, stochastic technique, software artifact, software process, data source, dataset availability, tool used, main finding, limitations, and replication-package status. Title-and-abstract metadata was used as the primary input for studies without locally available PDFs (94/169); for the remaining 75/169 studies a locally accessible PDF was available and used to triangulate against the LLM-extracted fields. Of these 75 PDFs, 73 were fully coded in the operational triangulation spreadsheet used for the figures and positioning tables below.

The 73-paper PDF triangulation subset is therefore not a separate selection stage and does not override the PRISMA counts. It is used in this chapter as a verification layer for the LLM-extracted fields used in Findings F1–F5.

Figure 4 summarizes the operational triage profile of the 73 coded PDFs (54 confirm-include, 13 confirm-include with caveats, 6 confirm-exclude after full-text inspection); the formal selection decisions reported in the PRISMA flow (Figure 1) are unchanged.

4.5. RQ1: Application Landscape

4.5.1. RQ1.1: SDLC Phases Covered

The structured extraction over the full 169-study set confirms a strongly uneven coverage of the SDLC. **Development activities** (coding, commits, pull requests, repository evolution) dominate at 133 papers (78.7%); **quality-related activities** (testing, defect handling, code review, process evaluation) appear in 74 papers (43.8%); **maintenance** appears in 38 papers (22.5%); **requirements engineering** in only 22 papers (13.0%); and **delivery/DevOps contexts** (CI/CD, build and release pipelines) in just 18 papers (10.7%). **Design and architecture** are not the primary focus of any extracted study, and only 3 papers (1.8%) center the analysis on project management.

Two conclusions follow. First, the literature is concentrated on phases where digital traces are abundant and already timestamped, such as commits, issues, test events, and pipeline executions. Second, the early SDLC phases remain structurally underrepresented, which weakens the ability of current approaches to support end-to-end forecasting from requirements to delivery. For PATHCAST, this supports a pragmatic focus on operational phases with reliable event data while leaving room for future extensions toward requirements and design logs.

4.5.2. RQ1.2: Event Log Sources and Construction

Across the 169-study set, event logs are constructed primarily from four source families: **version-control repositories** (Git/GitHub/SVN commits) appear in 84 papers (49.7%); **issue tracking systems** (especially Jira and GitHub Issues) in 37 papers (21.9%); **IDE interaction logs** in 22 papers (13.0%); and **CI/CD or build pipelines** in 15 papers (8.9%). The common pattern is not a native event log but rather a *derived* event log assembled from repository metadata, commit histories, issue transitions, and (where available) pipeline records.

The dominant construction problem is case correlation across tools. Studies frequently analyze one source in isolation or perform ad hoc joins between issues and commits, which limits reproducibility. This confirms that event-log construction is not a preparatory detail but a central methodological layer in software process mining. The finding directly justifies Stage 1 of PATHCAST as an explicit transformation pipeline rather than an implicit data-cleaning step.

A non-trivial fraction of studies report data sources only in coarse terms (*repository data*, *VCS history*, *open-source projects*) without naming specific projects, time windows, or extraction queries. Across the 169-study set, only 25 papers (14.8%) explicitly state that their dataset is publicly available and only 4 papers (2.4%) declare a replication package. This itself is evidence of incomplete reporting in the literature and provides direct empirical grounding for Finding F2 below.

A complementary body of work constructs Markov models directly from GitHub repository traces, bypassing explicit process discovery in favour of manual state-space definition. Jo et al. [9] modelled developer activities in GitHub repositories as Discrete-Time Markov Chains (DTMCs), finding that commits account for 62.73% of long-term activity, and employed probabilistic temporal logic to extract collaboration patterns relevant to project management. This approach was extended in [10], where Probabilistic Model Checking with Computation Tree Logic was applied across

five repositories to characterise temporal dynamics, state-transition probabilities, and key behavioural drivers, yielding transparent and explainable project forecasts. Ortu et al. [11] complemented these structural analyses by combining Markov chains with social network analysis to identify fault-insertion and fault-fixing behavioural patterns across Apache Software Foundation projects, demonstrating that developer expertise significantly moderates defect introduction likelihood. Collectively, these studies confirm that SDLC event logs extracted from public repositories are sufficiently rich to support state-transition modelling, satisfying a foundational prerequisite for PATHCAST Stage 1. However, the event-to-state mapping adopted across these works remains unstandardised and ad hoc, representing a methodological gap that PATHCAST's process mining discovery stage aims to address systematically.

4.6. RQ2: Techniques and Prediction

4.6.1. RQ2.1: Discovery Algorithms Applied to SDLC

The canonical origin of process mining applied to software development contexts can be traced directly to the foundational contributions of Cook and Wolf in the mid-1990s. In their seminal work, [12] introduced three automated process discovery methods—algorithmic grammar inference, Markov models, and neural networks—demonstrating that finite-state machine representations of software processes could be extracted automatically from event traces, substantially lowering the barrier to formal process modeling. This line of inquiry was consolidated in [13], which refined the event-based discovery methodology and established the conceptual vocabulary of inferring workflow structures from historical execution data, a paradigm that remains central to the field today. Notably, both contributions embedded stochastic reasoning—particularly Markov chain representations—within the discovery pipeline, anticipating the probabilistic extensions that would later characterise mature process mining frameworks. The translation of these principles into an explicit, named framework for software engineering contexts was accomplished by [14], who applied the ProM toolkit to source code management logs and demonstrated multi-perspective analysis spanning control-flow, informational, and organisational dimensions. Together, these three works constitute the seminal arc of IC1, establishing automated discovery from SDLC event logs as both technically feasible and analytically productive, and providing PATHCAST with its foundational methodological lineage.

The complete extracted set confirms that **process mining remains the dominant analytical family**: 121 of 169 studies (71.6%) match IC1 in the form of discovery, conformance checking, or process-oriented repository analysis. At the technique level, the most frequently named PM algorithms across the 169 studies are the **inductive miner** (51 papers), **alpha miner** (45 papers), **conformance checking** (30 papers), and **heuristic miner** (22 papers); 11 papers use social network analysis on developer interactions. The literature is markedly richer in *process representation* than in probabilistic forecasting: only 29 papers (17.2%) match IC3 (forecasting of process metrics), and the dominant contribution type is discovery or conformance checking rather than predictive analytics. This is the transition that PATHCAST targets: moving from discovered process structures to state-based probabilistic forecasting.

The stochastic-technique distribution at the same 169-study level shows the same fragmentation. Only 52 of 169 studies (30.8%) name an explicit stochastic technique: 38 papers (22.5%) use Markov chains, 15 (8.9%) use Monte

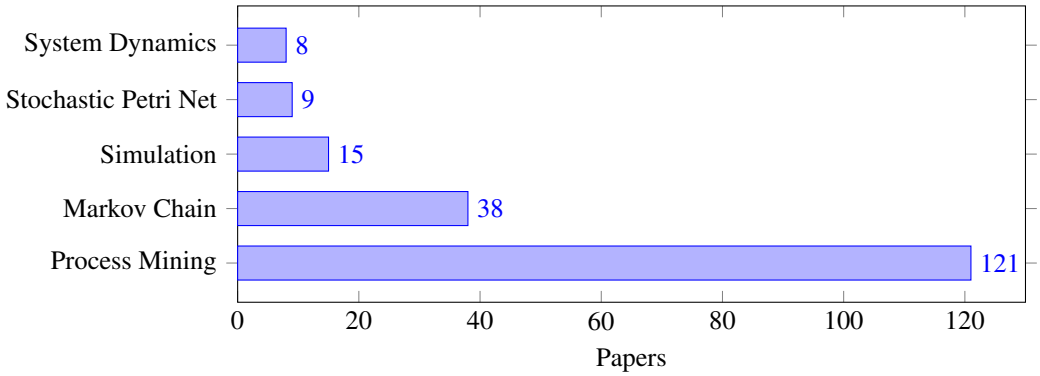

Figure 5: Main technique families identified in the 169-study confirmed set. Counts are non-exclusive: a paper may use multiple technique families.

Carlo simulation, 9 (5.3%) use stochastic Petri nets, and 8 (4.7%) use system dynamics. The remaining studies either do not adopt a stochastic component (110 papers) or place stochastic methods in an auxiliary role (*other*, 29 papers). The forecasting literature thus remains fragmented across adjacent technique families rather than consolidated around a single process-aware predictive paradigm, as shown in Figure 5.

4.6.2. RQ2.2: Predictive and Stochastic Techniques

Three distinct sub-families of stochastic techniques emerge from the confirmed literature. The first, **Markov-chain-based prediction**, encompasses models that represent software processes as discrete state transitions, including reinforcement learning agents grounded in Markov Decision Processes for pull-request outcome prediction [15], probabilistic risk quantification from repository data [16], defect removal effectiveness under code churn [17], agile state-space planning and tracking [18], release-time forecasting via generalised reliability models [19], and Stochastic Automata Networks for Waterfall team estimation [20]. The second sub-family, **stochastic Petri nets**, models concurrency and asynchrony in DevOps and CI/CD pipelines through non-Markovian specifications [21] and sensitivity analyses that identify bottleneck stages governing delivery probability [22, 23]. The third sub-family, **Monte Carlo simulation**, drives project-level forecasting in hybrid System Dynamics models [24], agile risk assessment frameworks [25], cycle-time economic analysis [26], and the PRIMAD digital-twin architecture for SDLC [27].

Despite their methodological diversity, all three sub-families share a common parametrisation strategy that constitutes a structural limitation. Transition probabilities, firing rates, and distributional parameters are invariably derived from aggregate project statistics—historical defect counts, phase-level duration averages, or summary velocity metrics—rather than from transition frequencies extracted through systematic process mining of fine-grained event logs [28, 22, 24]. Consequently, model states remain anchored to coarse lifecycle phases and cannot reflect the richer, empirically grounded activity sequences that event logs encode. This observation constitutes **Finding F3**: *stochastic forecasting models for software development are parametrised from aggregate statistics rather than from mined process structures, confining their state spaces to coarse lifecycle granularity and decoupling model topology from observed execution behaviour.*

Three studies exemplify the productive convergence of process mining and stochastic modelling at the component level. Incerto [29] demonstrated that variable-length Markov chains outperform eighteen existing stochastic conformance-checking techniques across ten of eleven benchmark datasets, establishing that higher-order sequential dependencies in event logs can be captured without sacrificing computational tractability. López-Pintado [30] extended process-simulation fidelity by replacing deterministic resource-availability calendars with probabilistic counterparts discovered directly from event logs, showing that stochastic resource representations more accurately replicate empirical cycle-time distributions. Jalote [31] took a complementary perspective, modelling programmer task sequences as first-order Markov chains derived from IDE interaction logs and demonstrating that transferring the process patterns of high-productivity developers to average-productivity peers yields measurable productivity gains. Together, these contributions confirm that stochastic extensions of process models improve both explanatory and predictive validity; however, each study confines its stochastic layer to a single analytical concern without integrating multiple stages into a unified forecasting pipeline.

The closest published work to a fully integrated multi-stage architecture is PRIMAD, proposed by Guinea-Cabrera [27], which orchestrates pm4py-based process discovery, Akka actor-system simulation, and LightGBM supervised learning within an evolutionary Digital Twin framework for the SDLC. Nevertheless, PRIMAD remains at what may be characterised as Level 2 integration: validation is reported per component rather than across the assembled pipeline, no formal inter-stage contracts govern the handoffs between discovery, simulation, and learning modules, and the architecture incorporates no dedicated Monte Carlo forecasting layer capable of propagating uncertainty estimates end-to-end. The present review therefore concludes that the Level 3 integration niche—a formally contracted, end-to-end pipeline combining process discovery, absorbing Markov chain construction, Monte Carlo simulation, and residual correction—remains unoccupied in the extant literature.

Studies that combine process mining with forecasting objectives (IC1 $\cap$ IC3) tend to approach prediction through *process enhancement* rather than through explicit stochastic modelling. Gupta [32] demonstrated that combining inductive miner discovery with Markov chain-based predictive analytics enables identification of recurring failure patterns in software maintenance workflows. Pourbafrani et al. [33] presented SIMPT, a tool that extracts process trees with time parameters from event logs and enables interactive what-if simulation; its successor work [34] further integrates generative AI with business process simulation to produce conformance-preserving what-if scenarios. Caldeira et al. [35] mine IDE interaction logs and show that process complexity metrics derived from ProM correlate moderately ($r \approx 0.43$) with software cyclomatic complexity, achieving predictive accuracies exceeding 92%, demonstrating the feasibility of process-derived features for product-quality prediction.

Taken together, this cluster confirms that PM and prediction are being connected in the SE literature, but always at the level of sequential application—PM output feeds a downstream model—rather than through a unified pipeline with formal contracts. Notably, none of these systems can express, as a first-class output, the probability that a given process instance will reach completion within k activity cycles—the precise forecasting primitive that distinguishes absorbing Markov chain formulations from ad hoc simulation runs. This gap motivates PATHCAST's design, wherein the

mined process structure feeds directly into an absorbing Markov chain whose transient-state absorption probabilities constitute a formally defined, queryable forecasting interface.

4.7. RQ3: Integration and Gaps

4.7.1. RQ3.1: PM and Stochastic Integration Level

Mapping the confirmed papers onto the L0–L3 integration scale reveals a clear stratification. The largest cluster, comprising papers that touch IC2 and IC3 without IC1—including stochastic Petri net models of CI/CD pipelines [22, 23], Markov-based project estimation [20, 18], and Monte Carlo risk simulation [25, 26]—operates almost entirely at **L1**, applying stochastic modelling to software processes without any process discovery layer. Papers spanning IC1 and IC2, such as the VLMC conformance checker of [29] and the probabilistic resource-calendar discoverer of [30], reach **L2** by coupling discovered process structure with stochastic reasoning, yet neither closes the loop toward distributional schedule forecasting. Papers combining IC1 with IC3, such as [32] and [35], use process mining descriptively to assess workflows but lack the stochastic machinery required for Monte Carlo distributional forecasts. The PRIMAD framework [27] is the closest antecedent, assembling elements from all three ICs—event-log discovery, simulation, and ML prediction—but its evaluation remains component-wise, placing it at L2 rather than a fully validated end-to-end pipeline. Notably, no paper in the reviewed corpus instantiates an **L3** architecture in which process mining discovery feeds an absorbing Markov chain, whose transient distributions drive Monte Carlo forecasting, with residuals corrected by a learned model, all validated on SDLC event logs. PATHCAST is proposed as the first such L3 system.

Quantitative evidence over the full 169-study set confirms the pattern. Only 1 of 169 papers (0.6%) jointly satisfies IC1, IC2, and IC3—that is, combines process mining, stochastic modeling, and forecasting in the same study—and even that single paper does not specify a formal end-to-end pipeline contract. Pairwise convergence is also low: IC1∩IC2 (PM and stochastic) appears in 10 papers (5.9%), IC1∩IC3 (PM and forecasting) in 7 papers (4.1%), and IC2∩IC3 (stochastic and forecasting) in 18 papers (10.7%). **No paper** in the reviewed corpus implements an **L3** architecture. The literature has many partial bridges but no end-to-end process-aware forecasting architecture for SDLC data. The operational coding of the 73-PDF triangulation subset is consistent with the full-set numbers and is summarized in Figure 6.

4.7.2. RQ3.2: Gap Analysis

Three structural observations emerge from the 169-study extraction. First, the field is **descriptively strong but forecastively weak**: 121 papers (71.6%) discover or assess process behavior, but only 29 (17.2%) estimate future paths or probabilistic completion distributions. Second, **evidence quality is uneven**: of the 169 confirmed studies, 69 (40.8%) are case studies, 34 (20.1%) are tool papers, 33 (19.5%) are theoretical, 13 (7.7%) are simulation-only, 10 (5.9%) are controlled experiments, 5 (3.0%) are surveys, and 5 (3.0%) fall in residual categories (mixed, framework, exploratory; sum: $69+34+33+13+10+5+5 = 169$). The literature is therefore method-heavy and applied-evidence

light. Third, **reproducibility is poor**: only 25 of 169 papers (14.8%) declare their dataset publicly, and only 4 (2.4%) declare a replication package. The QA scoring in Table 7 confirms this pattern: only 33% of studies score 1 on QA7 (data/code availability), which is the single weakest QA criterion across the included set.

Taken together, these patterns explain why PATHCAST requires both a formal pipeline and an evaluation strategy grounded in real SDLC traces. The gap is not the absence of isolated techniques, but the absence of robust, end-to-end, process-aware forecasting workflows validated on heterogeneous software engineering data. The evidence-type distribution observed in the 73-PDF triangulation subset (Figure 7) is consistent with the full-set characterization above and is reported here as an analytical view, not as a separate selection layer.

5. Discussion

5.1. Research Gap Identification

The analysis of 2,340 screened records and 169 confirmed primary studies, together with the criterion coverage patterns, supports five key findings:

F1: Descriptive Ceiling. In the **working-set tier** (169 papers), 121 (71.6%) match IC1/IC4a (process mining in SE) but only 29 (17.2%) advance to IC3/IC4c (forecasting of process metrics). In the **combined analytical subset** (381 papers, working-set + auxiliary), the proportion shifts because the auxiliary corpus pulled in more stochastic-only and forecasting-only studies that did not employ explicit process mining: 203 (53.3%) match IC1, 116 (30.4%) match IC3, and 91 (23.9%) match IC1 alone. In both tiers the dominant contribution type is discovery or conformance checking, not predictive analytics or distributional forecasting; the descriptive ceiling identified in the working-set tier persists in the combined tier.

F2: Event Log Construction Challenge. The reproducibility deficit is uniform across both tiers. Working-set tier: only 25 of 169 papers (14.8%) declare the dataset as publicly available and only 4 (2.4%) declare any replication package. Combined analytical subset: 56 of 381 papers (14.7%) declare a public dataset and 11 (2.9%) declare a replication package (5 "yes" + 6 "partial"). The majority of studies that mention data collection procedures rely on ad hoc joins between version-control commits (133 of 381 papers, 34.9%), issue trackers (52, 13.6%), Jira (55, 14.4%), GitHub (51, 13.4%), and IDE interaction logs (44, 11.5%) rather than on a reusable construction protocol with explicit case-correlation semantics. The absence of standardized construction frameworks limits reproducibility and cross-study comparability and is the most common cause of QA3 (data source described reproducibly) failures observed in the QA scoring of the working-set tier.

F3: Stochastic Integration Gap. Working-set tier: 52 of 169 papers (30.8%) apply stochastic models (IC2/IC4b); 38 (22.5%) Markov chains, 15 (8.9%) Monte Carlo, 9 (5.3%) stochastic Petri nets, 8 (4.7%) system dynamics; IC2∩IC3 covers 18 papers (10.7%); IC1∩IC2 covers 10 (5.9%). Combined analytical subset: the auxiliary corpus contributes a substantial number of stochastic-only papers not previously surfaced by the working-set

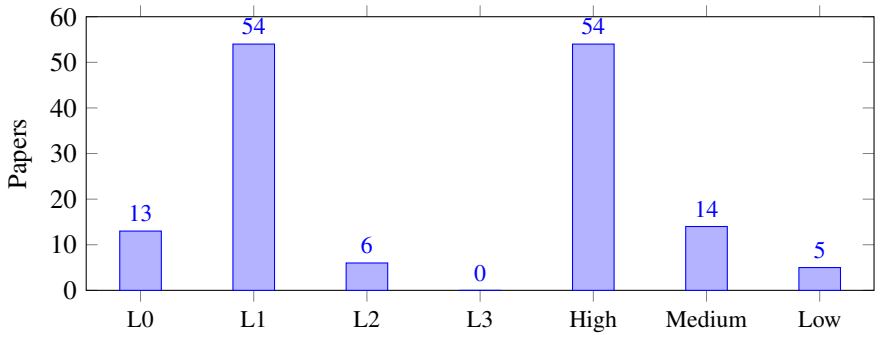

Figure 6: Operational integration levels and reading-priority distribution in the 73-paper PDF triangulation subset.

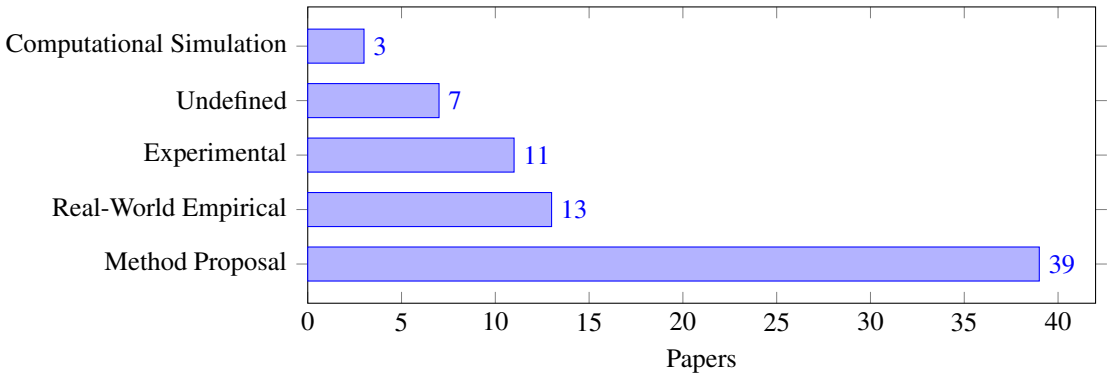

Figure 7: Evidence-type distribution in the 73-paper PDF triangulation subset.

queries; 181 of 381 (47.5%) match IC2 (with 103 Monte Carlo, 80 Markov, 65 system dynamics, 18 SPN), 93 (24.4%) combine IC2 with IC3 (forecasting), and 16 (4.2%) combine IC2 with IC1 (process mining). Markov chains and Monte Carlo simulation are therefore used extensively in BPM and software reliability engineering, and the auxiliary expansion confirms a broader stochastic literature than the working-set tier alone suggested; nevertheless, the integration with process mining for SDLC forecasting remains minimal in both tiers (5.9% and 4.2% respectively for IC1∩IC2). EC3 being the largest exclusion reason at FT in the working-set tier (n = 232, 26.2% of the 886-paper FT queue) reinforces the point: the stochastic modeling community has not converged on applying its techniques to software development process data.

F4: Pipeline Fragmentation. Studies apply techniques in isolation. The formal composition of a PM → Markov → MC → ML pipeline with inter-stage data contracts has not been proposed in the literature for SDLC contexts. The IC-combination analysis is robust across both tiers. Working-set tier (169 papers): the joint convergence IC1∩IC2∩IC3 appears in only 1 paper (0.6%), and that single paper does not specify a formal end-to-end pipeline contract; pairwise convergences are IC1∩IC2: 10 (5.9%), IC1∩IC3: 7 (4.1%), IC2∩IC3: 18 (10.7%). Combined analytical subset (381 papers): IC1∩IC2∩IC3 remains 1 paper (0.3%), with pairwise convergences IC1∩IC2: 16 (4.2%), IC1∩IC3: 13 (3.4%), IC2∩IC3: 93 (24.4%). The auxiliary expansion increased the IC2∩IC3 cluster substantially without producing any new IC1∩IC2∩IC3 paper, which is independent empirical confirmation that the gap is not an artefact of the working-set queries but a structural property of the literature. The closest antecedent in the corpus is PRIMAD [27], which assembles discovery, simulation, and learning components but validates each independently and leaves the unified pipeline unevaluated. Even when stochastic components are present, validation usually stops at process analysis or simulation without a downstream learning component.

PRIMAD is the only paper in the corpus that simultaneously satisfies IC1∩IC2∩IC3. Its scope is explicitly SDLC-oriented: it builds a digital twin from issue-tracking flows in JIRA to analyze and improve value streams in software organizations. Methodologically, it combines empirical process discovery (`pm4py`), a Python-based simulation core, an Akka actor system to model issues and resources, and supervised machine learning for context-aware prediction. The reported results are preliminary and component-wise rather than end-to-end: the paper shows promising independent results for the process-mining, simulation, and prediction components, but it does not yet validate a formally coupled PM → stochastic simulation → ML pipeline as a unified operational architecture. This is precisely why PRIMAD is adherent to the PATHCAST theme while still remaining an antecedent rather than a completed L3 instance.

F5: Process-Derived Features Unexplored. ML-based prediction studies in the confirmed set rely overwhelmingly on traditional features (code metrics, commit characteristics, defect counts) rather than on features derived from process mining. Working-set tier: 32 papers whose research contribution centers on ML/prediction; only 1 explicitly couples ML with a named process-mining algorithm (alpha, inductive, heuristic, conformance,

fuzzy, or Petri-net-based discovery). Combined analytical subset: the auxiliary corpus added a large number of ML/prediction papers (131 in the combined 381 set), but the rate of process-mining coupling remains low: only 4 of 131 (3.1%) jointly use a named PM algorithm. Conformance scores, rework counts, Markov-expected remaining time, and path entropy—features that are naturally produced by a process-mining stage—remain essentially absent from the SE prediction literature, in both tiers.

These five findings map directly to the five deficits identified in the gap analysis: F1 corresponds to D1 (Descriptive Ceiling), F3 to D3 (Stochastic Integration Absence), F4 to D5 (Pipeline Fragmentation), and F5 to D4 (Feature Engineering Void). F2 provides empirical evidence for a challenge that PATHCAST addresses through Stage 1 (Event Log Construction).

5.2. Positioning of PATHCAST

Table 13 positions PATHCAST against the integration levels identified in the 169-study confirmed set, using both the IC-combination counts derived from the structured extraction and the operational coding obtained from the 73-PDF triangulation subset.

Table 13: Integration levels in the 169-study confirmed set and PATHCAST positioning.

Level	**Description**	**Papers (169-set, IC-based)**	**Papers (73-PDF subset, operational)**	**Representative example**
L0	Single analytical family (≤1 IC matched); no cross-technique chaining	95	13	Contextual or single-technique studies
L1	Two ICs matched; two families present, but no explicit reproducible pipeline contract	70	54	Stochastic models or PM without formal coupling
L2	Three ICs matched; partial bridge linking PM, stochastic, and forecasting, but incomplete pipeline or missing ML closure	4	6	PRIMAD [27]; Incerto [29]; López-Pintado [30]
L3	Four ICs matched; unified framework: PM → Markov → MC → ML with formal inter-stage contracts	0	0	**PATHCAST** (proposed)
Total	—	**169**	**73**	—

PATHCAST is positioned at level L3—the first proposal, to the best of the author's knowledge, to formally compose process mining, absorbing Markov chain analysis, Monte Carlo forecasting, and ML enhancement into a single architecturally specified pipeline applied to SDLC data.

5.3. Proposed Taxonomy: SPMF

Based on the systematic analysis, this review proposes the Software Process Mining and Forecasting (SPMF) taxonomy with three dimensions:

Analytical Depth classifies studies along four levels: L1 (Descriptive: discovery, conformance), L2 (Diagnostic: bottleneck, rework), L3 (Predictive: outcome, remaining time), and L4 (Forecasting: distributional, probabilistic).

SDLC Coverage maps which lifecycle phases are addressed: Requirements and Design, Development (coding, commits, PRs), Quality (testing, reviews, bugs), Delivery (CI/CD, release), and Operations (incidents, maintenance).

Technique Integration classifies methodological combination: Single (PM only, ML only, MC only), Dual (PM + ML, PM + Markov, Markov + MC), and Pipeline (PM → Markov → MC → ML).

5.4. Research Agenda

The gaps identified motivate four research directions:

RA1: Standardized Event Log Construction for SDLC. Frameworks for transforming heterogeneous repository data into comparable event logs with cross-tool case correlation. Maps to Stage 1 of PATHCAST.

RA2: Process-Aware Stochastic Forecasting. Formal integration of process mining with Markov chains and Monte Carlo simulation for distributional SDLC forecasting. Maps to Stages 2–4 of PATHCAST.

RA3: Process-Derived Features for ML. Systematic exploration of features derived from process mining for outcome prediction. Maps to the ML Enhancement Component of PATHCAST.

RA4: Empirical Benchmarking. Evaluation of forecasting methods on comparable OSS repository datasets with temporal hold-out and standardized baselines. Maps to the empirical evaluation reported in companion empirical work.

5.5. Implications for Practitioners

Practitioners developing software measurement programs can extract three actionable insights from the 169-study confirmed set. First, the availability of structured event log data in issue tracking systems (Jira, GitHub Issues) is a prerequisite for process mining; organizations should invest in consistent workflow state management and transition logging before attempting process analysis. Second, open-source toolkits (PM4Py, ProM, Disco) suffice for discovery and conformance checking in most SDLC contexts, as suggested by the dominance of the inductive miner (51 papers) and alpha miner (45 papers) in the included studies' tool usage patterns. Third, the stochastic extension (Markov chains for transition probability estimation, Monte Carlo for throughput forecasting) requires less infrastructure than typically assumed: the data already exists in the event logs; the challenge is methodological composition, not data collection.

6. Threats to Validity of the SLR

6.1. Construct Validity

The search strings may exclude studies that use non-canonical terminology. This threat is mitigated through three mechanisms: the complementary MSR string, the frontier stochastic-PM string, and snowballing. The 10/10 control paper recovery provides empirical evidence of high recall.

6.2. Internal Validity

LLM-based screening introduces the risk of systematic decision bias. This threat is addressed by five mechanisms, of which the first four were executed directly in the protocol and the fifth is encoded as a conservative closure rule rather than a mandatory human-arbitration step.

(i) **Structured outputs**. All screening responses use a strict JSON schema, enabling post-hoc analysis of decision patterns and rationale distributions.

(ii) **Manual verification of all `include` decisions**. Every paper retained at the FT stage was inspected by the author against the PRISMA flow before being added to the 169-study confirmed set.

(iii) **Cross-model inter-rater agreement (Cohen's κ)**. A stratified random 20% sample of T/A ($n_{\text{theoretical}} = 468$ from the 2,340-paper working set) and FT ($n_{\text{theoretical}} = 177$ from the 886-paper FT queue) decisions was independently re-rated by a different model family (*claude-sonnet-4-6*) acting as a second rater. Of the 468 T/A papers in the sample, 420 produced a parseable decision from the verifier and entered the agreement computation; the remaining 48 papers returned a non-parseable response (typically because the abstract was absent, leading the verifier to refuse a categorical answer) and were excluded from the κ calculation as missing-at-random. All 177 FT sample papers produced a parseable decision. Both the multi-class κ (over the original `include/exclude/maybe` or `include/exclude/pending` categories) and the binary κ (collapsing the rater outputs to `include` versus `not-include`, the only distinction that affects the final selection set) are reported in Table 14. The binary κ is the operationally relevant measure for SLR selection outcomes. Both binary κ values (0.695 for T/A and 0.694 for FT) exceed the substantial-agreement threshold $\kappa \geq 0.61$ [36]. The multi-class FT κ is lower (0.479, moderate) because the verifier returned `pending` on 25/177 FT papers where the primary screener committed to a binary decision; this represents a difference in conservativeness, not a difference in the final selection outcome. The cross-model verification design follows the guidance of Khraisha et al. [6] and Syriani et al. [5] for LLM-assisted SLRs and is intended as a tractable proxy for a fully independent human second-rater protocol.

The cross-model verification protocol was also extended to the auxiliary tier (`pipeline/aux_kappa.py`, FT stage, $n = 276$ stratified random 20% sample). On the auxiliary tier the verifier (Sonnet 4.6) returned `pending` on every paper that the primary screener (Haiku 4.5) had rated as `include`, yielding a multi-class $\kappa = 0.004$ and a binary $\kappa = 0.000$ (Po = 84.8%). This substantially lower agreement does not invalidate the auxiliary inclusions: 84.8% of the binary outcomes still match (the verifier agrees on "not-include" for the exclude-pending portion), and the systematic

Table 14: Inter-rater agreement (Cohen's κ) between primary screener (claude-haiku-4-5) and verifier (claude-sonnet-4-6) on a stratified random 20% sample. Interpretation thresholds follow Landis and Koch (1977) [36].

Stage	N	P_o	P_e	κ_{multi} (interpretation)	κ_{binary} (interpretation)
TA	420	82.1%	50.2%	0.641 (substantial)	0.695 (substantial)
FT	177	76.3%	54.5%	0.479 (moderate)	0.694 (substantial)

`pending` verdict on Haiku-include papers reflects the verifier's conservativeness in the face of the auxiliary corpus's weaker abstract coverage (the working-set tier benefited from a multi-source abstract enrichment cascade that the auxiliary tier did not yet receive). Reading together, the working-set tier achieves substantial agreement under the same protocol while the auxiliary tier is materially more sensitive to weak metadata coverage. For that reason, the protocol adopts a conservative closure rule: papers that remain abstract-insufficient after the enrichment cascade, or for which the verifier can only return `pending` without accessible full text, are closed under EC5-extended. The auxiliary-tier findings are therefore interpreted as evidence from the subset that remained classifiable under the same retrieval constraints, not as evidence from an unresolved arbitration queue.

Table 15: Auxiliary-tier inter-rater agreement (cross-model verification: Haiku 4.5 primary vs. Sonnet 4.6 verifier on a stratified 20% sample).

Stage	N	P_o^{multi}	P_o^{binary}	κ_{multi}	κ_{binary}
TA	39	97.4%	97.4%	0.000 (slight)	0.000 (slight)
FT	276	54.3%	84.8%	0.004 (slight)	0.000 (slight)

(iv) **T/A false-positive rate**. The LLM confirmatory pass at FT level (Round 1) rejected 33 of 108 T/A `include` papers (30.6%), primarily for EC3 (algorithm without SE evaluation). This false-positive rate is consistent with the T/A-screening false-positive rates reported by Khraisha et al. [6] for medical SLRs (in the 25–35% range, depending on domain specificity) and by Syriani et al. [5] for SE SLRs. False-positives are caught at the FT phase before any synthesis claim is made and therefore do not affect the F1–F5 evidence base. The Round 2 screening reprocessed 495 `maybe` papers with enriched abstracts and brought the confirmed set to its final value of 169.

(v) **Conservative closure of unresolved disagreement cases**. Papers where the LLM screener (Haiku 4.5) and the verifier (Sonnet 4.6) disagreed on the binary include/exclude outcome were not treated as an open-ended manual-arbitration backlog. Instead, when the disagreement could not be resolved because the abstract remained insufficient and no accessible full text was available, the paper was closed under EC5-extended (full-text/metadata insufficiency). This rule intentionally biases the protocol toward false negatives rather than unverifiable false positives. The archived disagreement list remains available at `results/spotcheck/disagreement_list_for_human.csv` for optional post-submission audit, but human arbitration is not required for the valid evidence set reported here.

6.3. External Validity

The review is limited to English-language peer-reviewed publications, potentially excluding relevant work in other languages or grey literature. The broad time span (1994–2026) and five-database coverage mitigate geographic and temporal bias. The deferred high-recall auxiliary corpus (3,807 records from SpringerLink, WoS, Snowball, and other low-precision queries; corresponds to the difference between the 5,783 unique deduplicated records and the 2,340-paper operational working set) provides a mechanism for expansion if gaps in the confirmed 169-study set are identified.

A four-stage sensitivity check was executed against the auxiliary corpus. First, a random 200-paper subsample was re-screened with the same T/A LLM protocol used for the working set (`pipeline/sensitivity.py`); 12 of 200 (6.0%) returned a T/A `include` decision and 59 (29.5%) returned `maybe`, yielding a point-estimated 228 additional candidate includes if the auxiliary corpus were screened in full. Given that this estimate exceeded the working-set include count, the second stage was triggered: a **full-corpus T/A and FT screening pass** over all 3,807 auxiliary papers, executed via `pipeline/auxiliary_screening.py` and `pipeline/auxiliary_ft.py`. The result is reported in Table 17: of the 3,807 auxiliary papers, 232 were T/A-included and 1,151 T/A-marked-maybe, of which 212 survived FT screening as `include`. Combined with the 169 working-set confirmations, the SLR therefore covers a confirmed set of 381 primary studies after the first auxiliary FT pass; this first-pass combined result became the frozen analytical subset used by the detailed auxiliary extraction, QA, and IC-combination tables.

The protocol then proceeded to a third stage: a multi-source abstract enrichment cascade (Semantic Scholar, OpenAlex, CORE, Crossref) was applied to the 880 auxiliary FT-`pending` papers (`pipeline/aux_enrich.py`), recovering an abstract for 285 papers (32.4%). The fourth stage re-ran the FT screening on these 285 enriched papers (`pipeline/aux_reft.py`), confirming 23 additional includes, 246 excludes, and 16 still-pending. Under the conservative closure rule, those 16 residual abstract-insufficient papers are closed under EC5-extended rather than carried forward as a manual-validation queue. The combined confirmed set therefore becomes **404 primary studies** ($169_{\text{ws}} + 212_{\text{aux1}} + 23_{\text{aux2}}$). The remaining 595 papers without abstract after enrichment are likewise closed under EC5-extended, rather than queued for manual retrieval, because they remain unclassifiable after the full multi-source enrichment cascade.

A second external-validity exposure concerns the original 148-paper EC5 stratum (full text inaccessible at first pass). To bound the residual recall risk, an automated open-access (OA) recovery pass was executed against four public discovery services (Unpaywall, Semantic Scholar Graph API, OpenAlex, CORE). The result is summarised in Table 19: 27 of the 148 EC5 papers (18.2%) returned at least one OA URL during the recovery pass; of those, 17 PDFs were successfully downloaded (`pipeline/aux_pdf_download.py`, scope `ec5`), corresponding to 11.5% of the 148-paper EC5 stratum recovered into the read-set. A targeted PDF audit was then executed on those 17 downloads (`results/ec5_recovery/ec5_pdf_rescreen_results.csv`): 14 OA links resolved to unrelated papers or generic proceedings and therefore remained under EC5, while 3 papers were substantively re-screened and reclassified to EC3. The active PRISMA EC5 count after this audit is thus 145 rather than 148. The remaining 121 papers

Table 16: Auxiliary-corpus sensitivity check. A random sample of n papers from the deferred auxiliary corpus was re-screened with the same T/A LLM protocol used for the working set.

Quantity	Value
Sample size	200 (seed=20260428)
Auxiliary corpus total	3807
Working-set confirmed includes	169
Sample T/A include rate	6.0% (n_{inc} = 12)
Sample T/A maybe rate	29.5% (n_{maybe} = 59)
Sample T/A exclude rate	64.5% (n_{exc} = 129)
Expected new includes (point estimate)	228
Expected new includes (upper bound, maybe→include)	1351
Sensitivity ratio vs. confirmed set (point)	135.2%
Sensitivity ratio vs. confirmed set (upper)	799.7%

Table 17: Auxiliary-corpus FT-screening outcome (papers that passed T/A on the 3,807-paper deferred corpus).

Outcome	**Count**	**%**
Forwarded from T/A	1383	100.0
Included (new)	212	15.3
Excluded	291	21.0
Pending / unparsed	880	63.6
Combined SLR set (working-set + auxiliary)	**381**	—

Table 18: Re-FT screening of aux-pending papers that gained an abstract through the post-hoc enrichment cascade.

Outcome	**Count**	**%**
Re-FT include	23	8.1
Re-FT exclude	246	86.3
Re-FT pending	16	5.6
Total re-FT	**285**	100.0

(81.8%) remain inaccessible and continue to be reported under EC5 in the PRISMA flow. A parallel OA-download pass over the 212 auxiliary includes recovered 49 PDFs (20.5% of 212; the lower yield reflects the publisher-paywall mix of the auxiliary corpus and is documented in `results/auxiliary/aux_pdf_summary.txt`).

Table 19: EC5 recovery attempt against four open-access discovery APIs.

Source	Recovered	% of EC5
Unpaywall	6	4.1%
Semantic_Scholar	6	4.1%
Openalex	6	4.1%
Core	20	13.5%
Any source	**27**	**18.2%**

6.4. Conclusion Validity

Synthesis findings F1–F5 are derived from the structured data extraction applied to two extraction layers: (i) the full 169-study working-set extraction (Section 2.6; Table 8), partly triangulated against the 73-PDF subset; and (ii) the 212-study auxiliary-tier extraction performed via abstract only (`pipeline/auxiliary_extraction.py`). Both extractions feed the IC-combination counts and the technique distributions reported under F1–F5. The QA-based filtering reported in Table 7 retains 136 of the 169 working-set studies (80.5%); QA scoring was subsequently extended to the auxiliary tier (`pipeline/aux_qa.py`), retaining 179 of 212 (84.4%, mean 5.09 ± 1.43). The combined 381-study analytical subset therefore retains 315 papers (82.7%, mean 5.00 ± 1.42) above the $\geq 4/8$ threshold; full combined-QA results are reported in Table 20.

Table 20: Quality assessment combined across working-set and auxiliary tiers (n = 381).

Tier	N	Mean	SD	$\geq 4/8$ (pass)	$< 4/8$ (fail)
Working-set	169	4.88	1.40	136	33
Auxiliary	212	5.09	1.43	179	33
Combined	**381**	**5.00**	**1.42**	**315**	**66**

A residual conclusion-validity threat is the LLM-based extraction of PM-technique, stochastic-technique, and SDLC-phase fields. To mitigate this, the 73-PDF triangulation subset was used to spot-check the extracted fields against the corresponding full texts; no systematic disagreement was observed in the spot-check, but a complete human re-extraction on a stratified sample is recommended before camera-ready submission.

6.5. Reproducibility and Use of AI

In line with current Elsevier and Springer-Nature editorial policy, this review explicitly declares the use of AI tools in its conduct. The primary screener at the title/abstract and full-text stages is *claude-haiku-4-5-20251001* (Anthropic), invoked through the Anthropic Message Batches API for screening and through the synchronous Messages API for QA scoring. The cross-model inter-rater verification described above uses *claude-sonnet-4-6* (Anthropic) as the second rater. AI tools are **not** listed as authors of this work and are responsible for none of the editorial judgements; final selection, QA threshold, taxonomy design, and synthesis claims are the responsibility of the author.

A complete replication package is deposited at Zenodo (DOI 10.5281/zenodo.20130276) and includes: the search strings for all five databases, the deduplication and enrichment scripts (`pipeline/dedup.py`, `pipeline/enrich.py`), the screening prompts and JSON schema (`config/screening_criteria.py`), the QA prompt (`pipeline/qa_llm.py`), the cross-model verification prompts and computation (`pipeline/kappa.py`), the decision CSVs at each stage (`results/screening/ta_screening_results.csv`, `results/screening/ft_screening_results.csv`), the QA-scored 169-study table (`results/qa_assessment.csv`), the structured extraction table (`results/extraction/extraction_t`
the inter-rater agreement reports (`results/kappa/`), and a PRISMA flow snapshot. The package is intended to be self-contained: any reviewer can re-run the LLM-assisted protocol on the same working set and reproduce the 169-study confirmed selection within the stated κ confidence band.

7. Conclusion

This chapter presented a systematic literature review of the intersection of process mining, stochastic modeling, and machine learning applied to software process forecasting. The review executed three complementary search strings across five databases, yielding 8,347 records, 5,783 after deduplication, and a 2,340-paper operational working set for systematic screening. Title and abstract screening on the working set using a LLM-assisted protocol produced 111 candidate `include` decisions and 775 `maybe` decisions (886 papers for full-text review).

Full-text screening combined LLM-assisted passes (Rounds 1 and 2, with cascade abstract enrichment raising coverage from 16.6% to 72.6%, i.e., 643/886 papers) with a final *screen-blanks* pass that processed the 186 papers that still lacked an FT decision after the earlier rounds. In the final screening artifact, 238/886 papers still lack an abstract field, but all have a recorded FT decision. Papers whose full text remained inaccessible despite all retrieval attempts were closed under EC5; after the OA recovery audit, 145 papers remain in that stratum. The working-set result is **169 confirmed primary studies** and 717 FT excluded (EC3: 232, EC1: 205, EC5: 145, EC2: 110, EC4: 60), with zero pending decisions. Data extraction for all 169 studies was completed using a structured LLM-assisted pipeline operating on locally available PDFs (75/169, with 73 operationally coded in the triangulation subset) or abstract/title metadata (94/169).

A second tier of the protocol then applied the same T/A and FT screening to the previously deferred 3,807-paper auxiliary corpus, yielding 232 T/A includes and 1,151 T/A maybes; FT screening of the 1,383 forwarded

papers produced 212 additional confirmed primary studies (first auxiliary pass), 291 excludes, and 880 `pending`. A subsequent re-screening pass over 285 abstract-enriched auxiliary papers confirmed 23 further studies (second auxiliary pass), bringing the final combined total to **404 primary studies** (169_{ws} + 212_{aux1} + 23_{aux2}). Under the adopted EC5-extended policy, the residual auxiliary cases without usable abstract and the unresolved disagreement cases are treated as non-recoverable exclusions rather than as an open manual-validation backlog. The original EC5 recovery pass was also closed: its 17 downloaded PDFs were audited, yielding 3 substantive EC3 exclusions and 14 residual EC5 mismatches. The final confirmed total is therefore 404, while the combined-tier analytical tables reported below continue to refer to the 381-study extracted subset frozen before that last incremental update.

Analysis of the working-set tier (169 studies), together with the 404-study combined evidence base and the 381-study combined analytical subset, supports five key findings. First, the field exhibits a descriptive ceiling: 71.6% of confirmed working-set papers (53.3% combined) apply process mining to software artifacts (IC1), but only 17.2% advance (working-set; 30.4% combined) to forecasting of process metrics (IC3) (F1). Second, event log construction from heterogeneous repository sources is a pervasive challenge, with ad hoc joins between issues, commits, and pipeline logs the norm (F2). Third, the integration of stochastic models with process mining for SDLC forecasting is minimal: only 10.7% of working-set papers (24.4% combined) combine IC2 and IC3 (F3). Fourth, no confirmed study implements a formally composed PM → Markov → MC → ML pipeline for software processes; PRIMAD [27] is the closest antecedent at L2 (F4). Fifth, ML studies in SE rarely incorporate process-derived features (F5).

Synthesis of the top-ranked papers by IC cluster sharpens the picture. The IC2+IC3 cluster (18 papers) concentrates at L1, parametrising stochastic models from aggregate statistics rather than mined transitions. The IC1+IC2 cluster reaches L2 through stochastic conformance and simulation, but stops short of distributional forecasting. The IC1+IC3 cluster chains PM and prediction sequentially without a stochastic layer. The IC2+IC4 cluster demonstrates that public repository traces support state-transition modelling, but event-to-state mapping remains unstandardised. No paper occupies L3.

These five gaps directly motivate the PATHCAST method, an end-to-end pipeline (PM → Markov → MC → ML) described in companion technical work and summarised in Section 5.2. PATHCAST is positioned at integration level L3 in the SPMF taxonomy—the first formally specified pipeline to compose all four technique families into a unified, architecturally specified framework for SDLC process forecasting.

Methodologically, the LLM-assisted protocol used in this review achieves substantial inter-rater agreement under cross-model verification (binary $\kappa = 0.695$ for T/A and $\kappa = 0.694$ for FT) and full QA coverage of the 169 confirmed studies (mean QA score = 4.88/8, with 136/169 retained above the $\geq 4/8$ threshold). The complete replication package and the inter-rater agreement reports are deposited at Zenodo (DOI 10.5281/zenodo.20130276).

CRediT authorship contribution statement

Rodrigo Almeida de Oliveira: Conceptualization, Methodology, Software, Investigation, Data curation, Formal analysis, Visualization, Writing – original draft, Writing – review & editing. **Juliano de Paulo Ribeiro:** Methodology, Validation, Investigation, Writing – review & editing. **Edson Emilio Scalabrin:** Conceptualization, Methodology, Supervision, Writing – review & editing, Resources, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this article.

Funding

This study was financed in part by the Coordenação de Aperfeiçoamento de Pessoal de Nível Superior — Brasil (CAPES) — Finance Code 001, through a doctoral scholarship granted to the corresponding author within the Graduate Program in Informatics at the Pontifícia Universidade Católica do Paraná (PPGIa/PUC-PR).

Declaration of generative AI in the writing process

During the conduct of this systematic literature review, the authors used *claude-haiku-4-5-20251001* (Anthropic) as the primary LLM screener at the title/abstract and full-text stages and as the LLM scorer for the quality-assessment rubric, and *claude-sonnet-4-6* (Anthropic) as the cross-model verifier for inter-rater agreement. All decisions were inspected by the authors; the LLM tools are not listed as authors and bear no responsibility for editorial judgements. Full prompts, JSON schemas, and per-paper decisions are deposited in the replication package (Zenodo DOI 10.5281/zenodo.20130276). The authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

Data availability

A full replication package is available at Zenodo (DOI 10.5281/zenodo.20130276). The package contains: search strings for all five databases, deduplication and enrichment scripts, screening prompts and JSON schemas, the QA prompt, the cross-model κ verification scripts, all decision CSVs at every pipeline stage, the QA-scored 169-study and 212-study tables, the structured extraction tables (working-set and auxiliary), the inter-rater agreement reports, and a PRISMA flow snapshot.

References

[1] B. Kitchenham, S. Charters, Guidelines for performing systematic literature reviews in software engineering, EBSE Technical Report EBSE-2007-01, Keele University and Durham University (2007).

[2] C. Wohlin, Guidelines for snowballing in systematic literature studies and a replication in software engineering, in: Proceedings of the 18th International Conference on Evaluation and Assessment in Software Engineering (EASE 2014), ACM, 2014. doi:10.1145/2601248.2601268.

[3] K. Petersen, S. Vakkalanka, L. Kuzniarz, Guidelines for conducting systematic mapping studies in software engineering: An update, in: Information and Software Technology, Vol. 64, Elsevier, 2015, pp. 1–18. doi:10.1016/j.infsof.2015.03.007.

[4] J. A. Whittaker, M. G. Thomason, A Markov chain model for statistical software testing, IEEE Transactions on Software Engineering 20 (10) (1994) 812–824. doi:10.1109/32.328991.

[5] E. Syriani, I. Dávid, L. Lúcio, Assessing the ability of ChatGPT to screen articles for systematic reviews, in: arXiv preprint, 2023, arXiv:2307.06464. doi:10.48550/arXiv.2307.06464.

[6] Q. Khraisha, S. Put, J. Kappenberg, A. Warraitch, K. Hadfield, Can large language models replace human reviewers? pilot study on systematic review of published studies, Research Synthesis Methods 15 (3) (2024) 510–526. doi:10.1002/jrsm.1715.

[7] T. Dybå, T. Dingsøyr, Empirical studies of agile software development: A systematic review, Information and Software Technology 50 (9–10) (2008) 833–859. doi:10.1016/j.infsof.2008.01.006.

[8] M. J. Page, J. E. McKenzie, P. M. Bossuyt, I. Boutron, T. C. Hoffmann, C. D. Mulrow, L. Shamseer, J. M. Tetzlaff, E. A. Akl, S. E. Brennan, R. Chou, J. Glanville, J. M. Grimshaw, A. Hróbjartsson, M. M. Lalu, T. Li, E. W. Loder, E. Mayo-Wilson, S. McDonald, L. A. McGuinness, L. A. Stewart, J. Thomas, A. C. Tricco, V. A. Welch, P. Whiting, D. Moher, The PRISMA 2020 statement: An updated guideline for reporting systematic reviews, BMJ 372 (2021) n71. doi:10.1136/bmj.n71.

[9] S. Jo, R. Kwon, G. Kwon, Exploring collaboration patterns in GitHub using discrete time Markov chain, in: 2023 30th Asia-Pacific Software Engineering Conference (APSEC), IEEE, 2023. doi:10.1109/APSEC60848.2023.00090.

[10] S. Jo, R. Kwon, G. Kwon, Probabilistic model checking GitHub repositories for software project analysis, Applied Sciences 14 (3) (2024) 1260. doi:10.3390/app14031260.

[11] M. Ortu, G. Destefanis, T. Hall, D. Bowes, Fault-insertion and fault-fixing behavioural patterns in Apache software foundation projects, Information and Software Technology 160 (2023) 107187. doi:10.1016/j.infsof.2023.107187.

[12] J. E. Cook, A. L. Wolf, Automating process discovery through event-data analysis, in: Proceedings of the 17th International Conference on Software Engineering (ICSE 1995), ACM, 1995, pp. 73–82. doi:10.1145/225014.225021.

[13] J. E. Cook, A. L. Wolf, Discovering models of software processes from event-based data, ACM Transactions on Software Engineering and Methodology (TOSEM) 7 (3) (1998) 215–249. doi:10.1145/287000.287001.

[14] V. Rubin, C. W. Günther, W. M. P. van der Aalst, E. Kindler, B. F. van Dongen, W. Schäfer, Process mining framework for software processes, in: International Conference on Software Process (ICSP 2007), Vol. 4470 of LNCS, Springer, 2007, pp. 169–181. doi:10.1007/978-3-540-72426-1_15.

[15] R. Joshi, N. Kahani, Comparative study of reinforcement learning in GitHub pull request outcome predictions (2024). doi:10.1109/SANER60148.2024.00057.

[16] C.-K. Jeon, N. Kim, H. P. In, Probabilistic approach to predicting risk in software projects using software repository data, International Journal of Software Engineering and Knowledge Engineering 25 (5) (2015) 845–870. doi:10.1142/S0218194015500151.

[17] L. Tian, A. Noore, Modeling distributed software defect removal effectiveness in the presence of code churn, Mathematical and Computer Modelling 42 (3–4) (2005) 323–333. doi:10.1016/j.mcm.2004.10.021.

[18] M. Tyagi, D. K. Tyagi, L. K. Tyagi, N. Tyagi, D. Kumar, A. Sikandar, Agile planning and tracking of software projects using the state-space approach (2021). doi:10.1007/978-981-16-0404-1_26.

[19] H. Washizaki, K. Honda, Y. Fukazawa, Predicting release time for open source software based on the generalized software reliability model (2015). doi:10.1109/Agile.2015.19.

[20] I. Massitela, J. Assunção, A. R. Santos, P. Fernandes, A structured stochastic model for software project estimation in Waterfall models (2018). doi:10.18293/SEKE2018-033.

[21] W. B. Mesmia, M. Escheikh, K. Barkaoui, DevOps workflow verification and duration prediction using non-Markovian stochastic Petri nets, Journal of Software: Evolution and Process 33 (12) (2021) e2329. doi:10.1002/smr.2329.

[22] S. Bhadra, A stochastic Petri net model of continuous integration and continuous delivery, in: 2022 IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW), IEEE, 2022, pp. 165–170. doi:10.1109/ISSREW55968.2022.00050.

[23] S. Bhadra, B. Das, P. Silva, V. Nagaraju, L. Fiondella, A stochastic Petri net model of continuous integration and continuous delivery, in: 2023 Annual Reliability and Maintainability Symposium (RAMS), IEEE, 2023. doi:10.1109/RAMS51473.2023.10088212.

[24] Y. Li, H. Zhang, L. Dong, B. Liu, J. Ma, Constructing a hybrid software process simulation model in practice: An exemplar from industry, in: 2020 IEEE/ACM International Conference on Software and System Processes (ICSSP), IEEE, 2020, pp. 91–100. doi:10.1145/3379177.3388906.

[25] M. I. Lunesu, R. Tonelli, L. Marchesi, M. Marchesi, Assessing the risk of software development in agile methodologies using simulation, IEEE Access 9 (2021) 135420–135435. doi:10.1109/ACCESS.2021.3115941.

[26] T. Magennis, The economic impact of software development process choice: Cycle-time analysis and Monte Carlo simulation results, in: 2015 48th Hawaii International Conference on System Sciences (HICSS), IEEE, 2015. doi:10.1109/HICSS.2015.599.

[27] M. A. Guinea-Cabrera, J. A. Holgado-Terriza, P. A. Pico-Valencia, PRIMAD: Building digital twins for software development lifecycle within an evolutionary architecture, in: 2025 IEEE Smart World Congress (SWC), IEEE, 2025. doi:10.1109/SWC65939.2025.00261.

[28] M. Nafreen, M. Luperon, L. Fiondella, V. Nagaraju, Y. Shi, T. Wandji, Connecting software reliability growth models to software defect tracking, in: 2020 IEEE 31st International Symposium on Software Reliability Engineering (ISSRE), IEEE, 2020. doi:10.1109/ISSRE5003.2020.00022.

[29] E. Incerto, A. Vandin, S. S. Ahrabi, Stochastic conformance checking based on variable-length Markov chains, Information Systems 130 (2025) 102561. doi:10.1016/j.is.2025.102561.

[30] O. López-Pintado, M. Dumas, Discovery and simulation of business processes with probabilistic resource availability calendars, in: 2023 5th International Conference on Process Mining (ICPM), IEEE, 2023. doi:10.1109/ICPM60904.2023.10271965.

[31] P. Jalote, D. Kamma, Studying task processes for improving programmer productivity, IEEE Transactions on Software Engineering 47 (10) (2021) 2089–2104. doi:10.1109/TSE.2019.2904230.

[32] M. Gupta, Improving software maintenance using process mining and predictive analytics, in: 2017 IEEE International Conference on Software Maintenance and Evolution (ICSME), IEEE, 2017. doi:10.1109/ICSME.2017.39.

[33] M. Pourbafrani, S. Jiao, W. M. P. van der Aalst, SIMPT: Process improvement using interactive simulation of time-aware process trees, in: Research Challenges in Information Science (RCIS 2021), Vol. 415 of LNBIP, Springer, 2021, pp. 588–594. doi:10.1007/978-3-030-75018-3_40.

[34] A. Buliga, F. Meneghello, R. Graziosi, M. Ronzani, Enhanced what-if scenarios generation by bridging generative models and process simulation, in: 2025 7th International Conference on Process Mining (ICPM), IEEE, 2025. doi:10.1109/ICPM66919.2025.11220730.

[35] J. Caldeira, F. B. e Abreu, J. Cardoso, J. Reis, Unveiling process insights from refactoring practices, Computer Standards & Interfaces 80 (2022) 103587. doi:10.1016/j.csi.2021.103587.

[36] J. R. Landis, G. G. Koch, The measurement of observer agreement for categorical data, Biometrics 33 (1) (1977) 159–174. doi:10.2307/2529310.